

Reusable Code

Since this is a playing card–based game, the code obviously creates and manipulates a simulated deck of cards. If we wanted to write another card-based game, it would be great to be able to reuse the code for the deck and cards.

In a procedural program, it can often be difficult to identify all the pieces of code associated with one portion of the program, such as the deck and cards in this example. In [Listing 1-1](#), the code for the deck consists of two tuple constants, two functions, some main code to build a global list that represents the starting deck of 52 cards, and another global list that represents the deck that is used while the game is being played. Further, notice that even in a small program like this, the data and the code that manipulates the data might not be closely grouped together.

Therefore, reusing the deck and card code in another program is not that easy or straightforward. In Chapter 12, we will revisit this program and show how an OOP solution makes reusing code like this much easier.

Bank Account Simulations

In this second example of procedural coding, I'll present a number of variations of a program that simulates running a bank. In each new version of the program, I'll add more functionality. Note that these programs are not production-ready; invalid user entries or misuse will lead to errors. The intent is to have you focus on how the code interacts with the data associated with one or more bank accounts.

To start, consider what operations a client would want to do with a bank account and what data would be needed to represent an account.

Analysis of Required Operations and Data

A list of operations a person would want to do with a bank account would include:

- Create (an account)